VIT BHOPAL
www.vitbhopal.ac.in

GRAPH ALGORITHMS VISUALIZER

Course: Discrete Mathematics and Graph theory

Course Code : MAT2002

Slot : B11 + B12 + B13

Faculty : Juhi Kesarwani

Group Members : Nehal Garg – 25BCE10492

Shreiya – 25BCE11110

Ananthi T – 25MIP10056

INTRODUCTION

Discrete Mathematics provides the mathematical foundation required to solve computational problems using logical structures. One of the most important topics in discrete mathematics is graph theory, which helps represent relationships between objects in the form of nodes and connections.

Many real-world systems such as road networks, communication systems, and computer networks can be modelled using graphs. Understanding how to find efficient paths and optimal connections in such networks is an essential problem in computer science.

In this project, we implemented and visualized graph algorithms to understand how theoretical concepts of discrete mathematics work in practice. The project focuses mainly on finding the shortest path between nodes and constructing a minimum-cost network using graph algorithms.

OBJECTIVES

The main objectives of this project are:

- **To understand graph representation using vertices and edges**
- **To apply discrete mathematics concepts through programming**
- **To implement shortest path calculation using Dijkstra's Algorithm**
- **To construct Minimum Spanning Tree using Kruskal's Algorithm**
- **To visualize algorithm results for better understanding**
- **To connect mathematical theory with real-world applications**

FUNDAMENTAL CONCEPT

Graph :

A graph is a structure consisting of

- **Vertices (Nodes) → represent objects or locations**
- **Edges → represent connections between nodes**

In this project, graphs are used to represent networks where nodes are connected using weighted edges.

Example: Cities connected by roads with distances.

Weighted Graph :

A weighted graph is a graph in which each edge has a numerical value called weight.

In our project:

- **Weight represents distance or cost between nodes.**
- **Algorithms use these weights to calculate optimal paths**

Minimum Spanning Tree (MST) :

A Minimum Spanning Tree is a subset of edges that:

- **Connects all vertices**
- **Contains no cycles**
- **Has minimum total edge weight**

The MST ensures that all nodes remain connected with minimum cost.

ALGORITHMS IMPLEMENTED

Dijkstra's Algorithm

Dijkstra's Algorithm is used to find the shortest distance from a starting node to all other nodes in a weighted graph.
Instead of checking all possible paths, the algorithm always selects the path with the smallest known distance, making it efficient.

Steps Followed:

1. **Assign distance 0 to the starting node.**
2. **Assign infinite distance to other nodes.**
3. **Select the node with minimum distance.**
4. **Update distances of neighboring nodes.**
5. **Repeat until all nodes are visited.**

Output:

- **Shortest distance from source node**
- **Optimal path selection**

Applications:

- **Navigation systems**
- **Network routing**

Kruskal’s Algorithm

Kruskal’s Algorithm is used to construct the Minimum Spanning Tree of a graph.
The algorithm selects edges in increasing order of weight while ensuring no cycles are formed.

Steps Followed:

1. **Sort all edges based on weight.**
2. **Select the smallest edge.**
3. **Add edge if it does not create a cycle.**
4. **Continue until all vertices are connected.**

Output:

- **Minimum cost network connecting all nodes.**

Applications:

- **Cable network design**
- **Road planning**
- **Resource optimization**

METHODOLOGY

The workflow of the project is shown below:

Graph Creation → Algorithm Execution → Result Calculation → Visualization

Process Description:

- **Graph data is created using nodes and weighted edges.**
- **Selected algorithm processes the graph.**
- **Mathematical calculations determine optimal results.**
- **Visualization displays the graph and algorithm output.**

RESULTS AND OBSERVATIONS

The project successfully demonstrated how graph algorithms work practically.

Observations include:

- **Shortest paths were computed correctly using Dijkstra's Algorithm.**
- **Kruskal's Algorithm produced a minimum spanning tree with minimum total weight.**
- **Visualization made algorithm behavior easier to understand.**
- **Practical implementation strengthened understanding of graph theory concepts.**

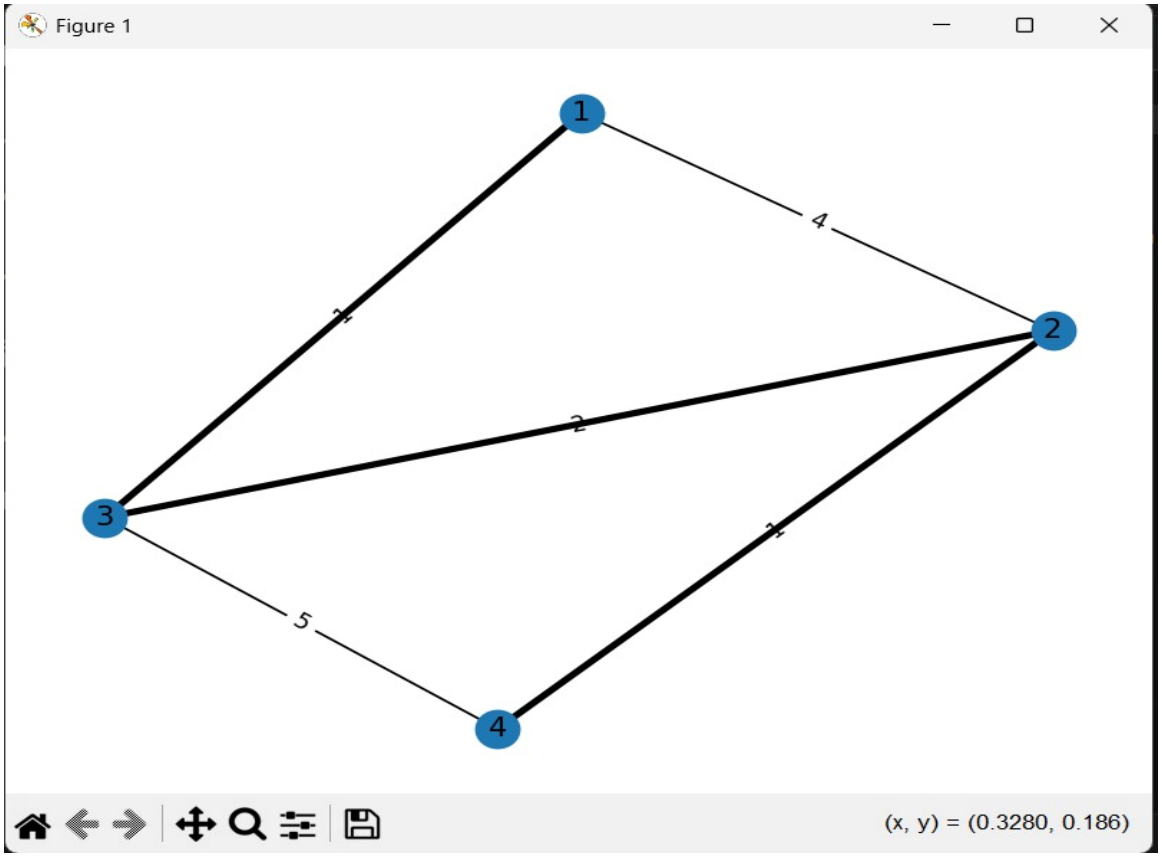
OUTPUTS :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

                              > python -u "c:\Users\Shreiya\Downloads\src\main.py"
Shortest Paths from Node 1:
{1: 0, 2: 3, 3: 1, 4: 4}

Minimum Spanning Tree (Kruskal):
[(1, 3, 1), (2, 4, 1), (3, 2, 2)]

Minimum Spanning Tree (Prim):
[(1, 3, 1), (3, 2, 2), (2, 4, 1)]
PS C:\Users\Shreiya\Downloads\src>
```



ADVANTAGES AND LIMITATIONS

Advantages

- **Simplifies understanding of graph algorithms**
- **Visual representation improves learning**
- **Demonstrates real application of discrete mathematics**

Limitations

- **Works on predefined graphs only**
- **Performance may reduce for very large networks**

CONCLUSION

This project helped in understanding how discrete mathematics concepts such as graphs and optimization algorithms are applied in real computational problems. By implementing Dijkstra's and Kruskal's algorithms, theoretical knowledge was transformed into practical experience.

APPLICATION

- **Route planning systems**
- **Internet data routing**
- **Network infrastructure design**
- **Transportation systems**
- **Optimization problems in computing**

TOOLS AND TECHNOLOGIES

The project was developed using:

- **Python – for algorithm implementation**
- **NetworkX Library – for graph representation**
- **Matplotlib – for visualization of graphs**
- **GitHub – for project collaboration and version control**

FUTURE SCOPE

- **Interactive graph input by users**
- **Web-based visualization interface**
- **Step-by-step animation of algorithms**
- **Support for larger datasets**

REFRENCES

- **Course Notes – Discrete Mathematics & Graph Theory**
- **Python Documentation**
- **NetworkX Official Documentation**